2026-7-22 Linux文件描述符、重定向与缓冲：从系统调用到 Stdio

- 1. 从 C 库函数下降到 Linux 系统调用
  - 1.1 为什么语言库不是文件操作的最底层
    - 1.1.1 文件最终由操作系统管理
    - 1.1.2 用位图同时传递多个打开标志
  - 1.2 系统调用 open
    - 1.2.1 open 的返回值与参数
    - 1.2.2 常用打开标志
    - 1.2.3 mode 还要和 umask 一起计算
  - 1.3 系统调用 write、read 与 close
    - 1.3.1 write 只写指定字节数
    - 1.3.2 read 读取的是字节，不会自动加 '\0'
    - 1.3.3 close 结束文件描述符的使用
  - 1.4 语言层接口与系统调用层的对应关系
    - 1.4.1 FILE 对象内部保存了文件描述符和缓冲区
    - 1.4.2 一次语言层写入最终仍要经过系统调用
    - 1.4.3 从 FILE 看到文件描述符
    - 1.4.4 语言库封装的意义：跨平台与效率
- 2. 文件描述符与进程打开文件表
  - 2.1 文件描述符为什么是一个整数
    - 2.1.1 fd 是进程文件描述符表中的下标
    - 2.1.2 进程与打开文件的关系
  - 2.2 文件描述符的分配规则
    - 2.2.1 总是选择最小可用编号
    - 2.2.2 三个默认描述符也遵循同一规则
- 3. 重定向的本质
  - 3.1 关闭后重新打开为什么能改变输出位置
    - 3.1.1 stdout 只记住数字 1
    - 3.1.2 覆盖与追加只相差打开标志
  - 3.2 用 dup2 复制文件描述符
    - 3.2.1 dup2 建立两个编号到同一目标的关联
    - 3.2.2 输出重定向
    - 3.2.3 输入重定向
    - 3.2.4 追加重定向
  - 3.3 Shell 重定向与标准错误
    - 3.3.1 Shell 先完成重定向，再执行目标程序

        - 3.3.2 标准错误存在的意义
- 4. 理解“一切皆文件”
    - 4.1 统一抽象让不同设备拥有相似接口
        - 4.1.1 普通文件与设备都可以被打开
        - 4.1.2 内核用 struct file 描述一次打开关系
- 5. 缓冲区的完整数据路径
    - 5.1 一次 I/O 为什么会有多次数据拷贝
        - 5.1.1 读取路径
        - 5.1.2 写入路径
        - 5.1.3 修改文件也要遵循“加载、修改、刷新”
        - 5.1.4 用户缓冲区由程序自己提供
    - 5.2 用户缓冲区、语言级缓冲区与内核缓冲区
        - 5.2.1 三种缓冲区的区别
        - 5.2.2 为什么要减少系统调用
        - 5.2.3 fopen 与 fclose 在缓冲链路中的位置
    - 5.3 标准 I/O 的三种缓冲模式
        - 5.3.1 全缓冲
        - 5.3.2 行缓冲
        - 5.3.3 无缓冲
        - 5.3.4 常见刷新时机
    - 5.4 三个典型实验
        - 5.4.1 只关闭 fd，没有刷新语言缓冲区
        - 5.4.2 _exit 会跳过标准 I/O 清理
        - 5.4.3 fork 后为什么会出现重复输出
    - 5.5 从内核缓冲区到持久化存储
        - 5.5.1 fflush 与 fsync 不是同一层操作
- 6. 手写一个精简版 stdio
    - 6.1 设计目标
        - 6.1.1 用 myFILE 复现“描述符 + 缓冲区”
        - 6.1.2 头文件 mystdio.h
    - 6.2 实现核心接口
        - 6.2.1 打开文件
        - 6.2.2 刷新缓冲区
        - 6.2.3 写入数据
        - 6.2.4 关闭文件
    - 6.3 验证缓冲区行为
- 7. 本章总结

> **本章定位**
>
> 上一章已经回答了“文件是什么”和“C 标准库怎样读写文件”。本章继续追问：文件和硬件最终由谁管理，语言库怎样落到系统调用，文件描述符为什么能代表打开的文件，重定向为什么能改变输出位置，以及缓冲区为什么能提高 I/O 效率。

> 笔记主线是：进程先通过文件描述符找到打开的文件；语言接口和系统调用都要把数据交给内核；内核再通过文件对象和缓冲区把数据送到外部设备。缓冲区不是孤立的概念，而是连接进程、操作系统和硬件的中间层。

1. 从 C 库函数下降到 Linux 系统调用

1.1 为什么语言库不是文件操作的最底层

1.1.1 文件最终由操作系统管理

应用程序运行在用户态。普通进程不能绕过操作系统直接操作磁盘控制器，也不能随意指挥网卡、显示器和键盘。因此，虽然 C 程序表面调用的是 `fopen`、`fread`、`fwrite`，底层仍然要依赖操作系统提供的系统调用。

可以把两类接口对应起来理解：

语言层接口	典型系统调用	主要职责
`fopen`	`open`	请求操作系统打开或创建文件
`fread`	`read`	把数据从内核空间拷贝到用户空间
`fwrite`	`write`	把数据从用户空间交给内核
`fclose`	`close`	关闭文件描述符并释放内核侧的引用
`fseek`	`lseek`	修改打开文件描述中的读写位置

语言库还要额外维护缓冲区、错误标志和格式化处理等状态，所以它并不只是把系统调用换一个名字，而是在系统调用外面又包了一层更易用的接口。

1.1.2 用位图同时传递多个打开标志

`open` 需要同时接受“只写、创建、清空、追加”等选项，这些选项会被编码成互不冲突的二进制位。多个标志可以通过按位或 `|` 组合，再通过按位与 `&` 检测。

```
#define ONE   (1 << 0)
#define TWO   (1 << 1)
#define THREE (1 << 2)
#define FOUR  (1 << 3)

void print_flags(int flags)
{
    if (flags & ONE)
        printf("one\n");
    if (flags & TWO)
        printf("two\n");
    if (flags & THREE)
        printf("three\n");
    if (flags & FOUR)
        printf("four\n");
}

int main(void)
{
    print_flags(ONE | TWO | FOUR);
    return 0;
}
```

一个二进制位表示一种独立信息：

- `|` 用于把一个或多个标志位设置为 1；
- `&` 用于判断某个标志位是否为 1。

Linux 的 `O_WRONLY`、`O_CREAT`、`O_TRUNC`、`O_APPEND` 等打开标志采用的就是同样的组合方式。

```
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ./openfile
one

one
two

one
two
three

one
two
three
four

one
four

ZL@VM-0-14-ubuntu:~/code/lesson13-file$
```

1.2 系统调用 `open`

1.2.1 `open` 的返回值与参数

最基本的函数形式可以概括为：

```
int open(const char *pathname, int flags, mode_t mode);
```

其中：

- `pathname` 是路径；
- `flags` 描述访问方式；
- `mode` 主要在创建新文件时指定初始权限；
- 成功时返回一个非负的文件描述符；
- 失败时返回 `-1`，同时设置 `errno`。

如果文件已经存在，以只读方式打开只需要两个参数：

```
int fd = open("log.txt", O_RDONLY);
```

如果文件不存在时还要创建，就需要加入 `O_CREAT`，并提供权限模式：

```c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    int fd = open("log.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0666);
    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    close(fd);
    return 0;
}
```

```
int open(const char *pathname, int flags, ...
                /* mode_t mode */ );
```

```
RETURN VALUE
       On success, open(), openat(), and creat() return the new file descriptor (a nonneg-
       ative integer).  On error, -1 is returned and errno is set to indicate the error.
```

1.2.2 常用打开标志

标志	含义
`O_RDONLY`	只读打开
`O_WRONLY`	只写打开
`O_RDWR`	可读可写打开
`O_CREAT`	文件不存在时创建
`O_TRUNC`	文件存在时截断为 0 字节
`O_APPEND`	每次写入前把位置放到文件末尾

可以组合出几种常见语义：

```
int fd1 = open("a.txt", O_RDONLY);
int fd2 = open("b.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
int fd3 = open("c.txt", O_WRONLY | O_CREAT | O_APPEND, 0666);
```

它们分别对应“只读”“覆盖写”“追加写”。`O_TRUNC` 与 `O_APPEND` 是很多重定向行为的底层差异来源。

1.2.3 `mode` 还要和 `umask` 一起计算

使用 `0666` 创建文件时，最终权限不一定是 `rw-rw-rw-`。进程还有一个权限掩码 `umask`，实际创建权限通常为：

```
mode & ~umask
```

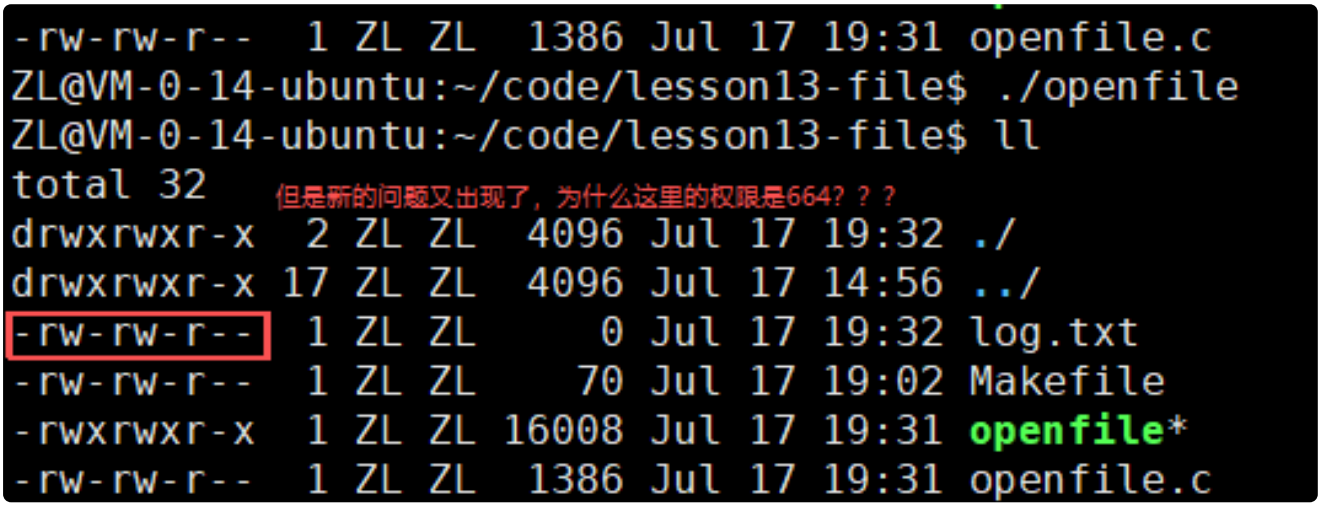
如果 `umask` 是 `0002`，则：

```
0666 & ~0002 = 0664
```

因此文件最终可能显示为 `rw-rw-r--`。`umask` 是进程级设置，可以在程序内调用 `umask()` 修改当前进程后续创建文件的默认掩码：

```
umask(0);
```

这会影响当前进程创建的新文件，不应把它理解为永久修改了系统中所有进程的配置。



1.3 系统调用 `write`、`read` 与 `close`

1.3.1 `write` 只写指定字节数

```
ssize_t write(int fd, const void *buf, size_t count);
```

`write` 从 `buf` 开始向文件描述符 `fd` 写最多 `count` 个字节。返回值表示实际写入的字节数；失败时返回 `-1`。它没有“自动补字符串结束符”的语义。

```
const char *msg = "asdf";
ssize_t n = write(fd, msg, strlen(msg));
```

这里使用 `strlen(msg)`，不把末尾的 `'\0'` 写入。`'\0'` 是 C 语言字符串的约定；文件本身保存的是一段字节，是否把该字节写进去由程序决定。

```
SYNOPSIS
       #include <unistd.h>

       ssize_t write(int fd, const void buf[.count], size_t count);
```



如果以 `O_WRONLY | O_CREAT` 打开一个已有文件，并且不指定 `O_TRUNC`，文件原内容不会被自动清空。写入较短的内容时，只会覆盖从当前文件位置开始的一部分字节。

若要模拟 C 库的 `"w"`：

```
int fd = open("log.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
```

若要模拟 C 库的 `"a"`：

```
int fd = open("log.txt", O_WRONLY | O_CREAT | O_APPEND, 0666);
```

1.3.2 `read` 读取的是字节，不会自动加 `'\0'`

```
ssize_t read(int fd, void *buf, size_t count);
```

返回值的含义是：

- 大于 0：本次实际读取的字节数；
- 等于 0：已经到达文件末尾；
- 小于 0：读取失败。

如果随后把缓冲区当作 C 字符串处理，需要自己预留终止符的位置：

```
char buffer[1024];
ssize_t n = read(fd, buffer, sizeof(buffer) - 1);
if (n > 0)
{
    buffer[n] = '\0';
    printf("%s\n", buffer);
}
```

1.3.3 `close` 结束文件描述符的使用

```
close(fd);
```

关闭文件描述符后，该描述符对应的表项就可以重新分配。继续使用已经关闭的 `fd` 会失败，不能假设它仍然指向原来的文件。

1.4 语言层接口与系统调用层的对应关系

1.4.1 `FILE` 对象内部保存了文件描述符和缓冲区

可以把 C 标准库的 `FILE` 理解成一个比系统调用更“厚”的对象，它通常包含：

1. 文件描述符 `fd`；
2. 输入或输出缓冲区；
3. 当前缓冲区位置；

4. 打开状态、错误状态、缓冲策略等信息。

概念上可以表示成：

```
typedef struct FILE
{
    int fd;
    char *inbuffer;
    char *outbuffer;
    size_t pos;
    /* 其他状态字段 */
} FILE;
```

不同 C 标准库的 `FILE` 实现并不要求完全相同，但“文件描述符 + 缓冲区 + 状态信息”这个结构是理解其行为的关键。

1.4.2 一次语言层写入最终仍要经过系统调用

对于多次小规模写入，C 库可能先把数据放进语言级缓冲区；当缓冲区满足刷新条件时，才调用一次 `write(fd)` 把积攒的数据交给内核。

因此，`printf`、`fprintf`、`fputs` 和 `write` 可以输出到同一个位置，但它们所处的抽象层不同：

- `printf`、`fprintf`、`fputs` 是 C 标准库接口；
- `write(1, ...)` 是直接面向文件描述符 1 的系统调用接口。

```
drwxrwxr-x  2 ZL ZL  4096 Jul 17 20:59 ./
drwxrwxr-x 17 ZL ZL  4096 Jul 17 14:56 ../
-rw-rw-r--  1 ZL ZL    70 Jul 17 19:02 Makefile
-rwxrwxr-x  1 ZL ZL 16008 Jul 17 20:59 openfile*
-rw-rw-r--  1 ZL ZL  1759 Jul 17 20:59 openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ./openfile
hhhhhhh
```

1.4.3 从 `FILE` 看到文件描述符

程序启动时，C 运行时通常会把标准流和三个默认文件描述符对应起来：

文件描述符	标准流	典型含义
0	`stdin`	标准输入
1	`stdout`	标准输出
2	`stderr`	标准错误

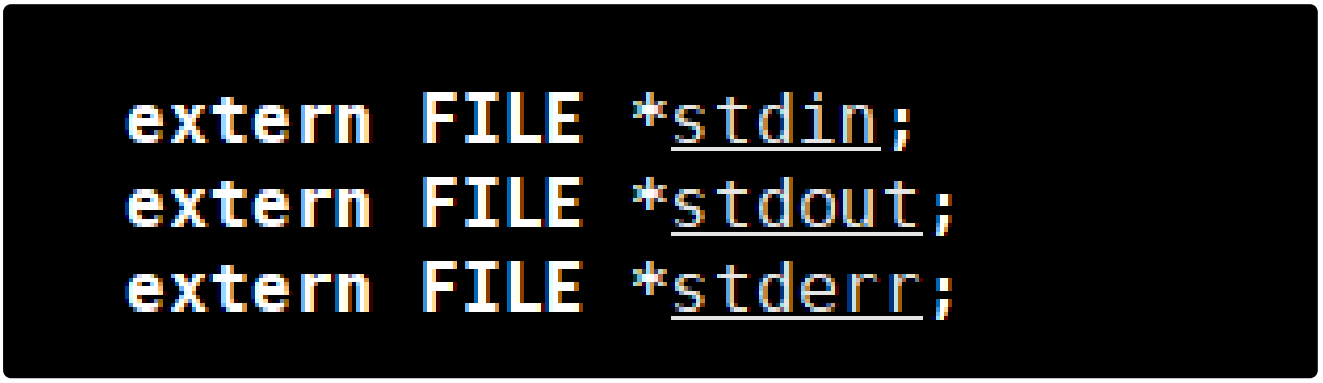
在 glibc 中，可以观察 `FILE` 内部的 `_fileno` 字段：

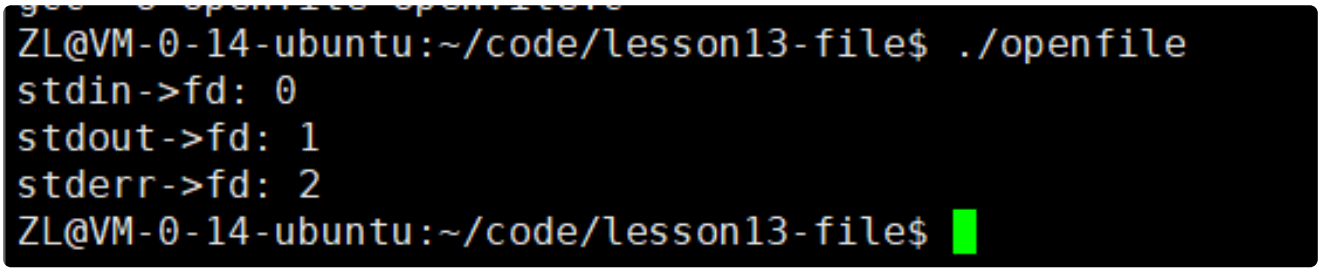
```c
#include <stdio.h>

int main(void)
{
    printf("stdin  -> fd: %d\n", stdin->_fileno);
    printf("stdout -> fd: %d\n", stdout->_fileno);
    printf("stderr -> fd: %d\n", stderr->_fileno);
    return 0;
}
```

`_fileno` 是 glibc 的具体字段，跨平台代码更适合使用 `fileno(fp)`：

```c
int fd = fileno(stdout);
```



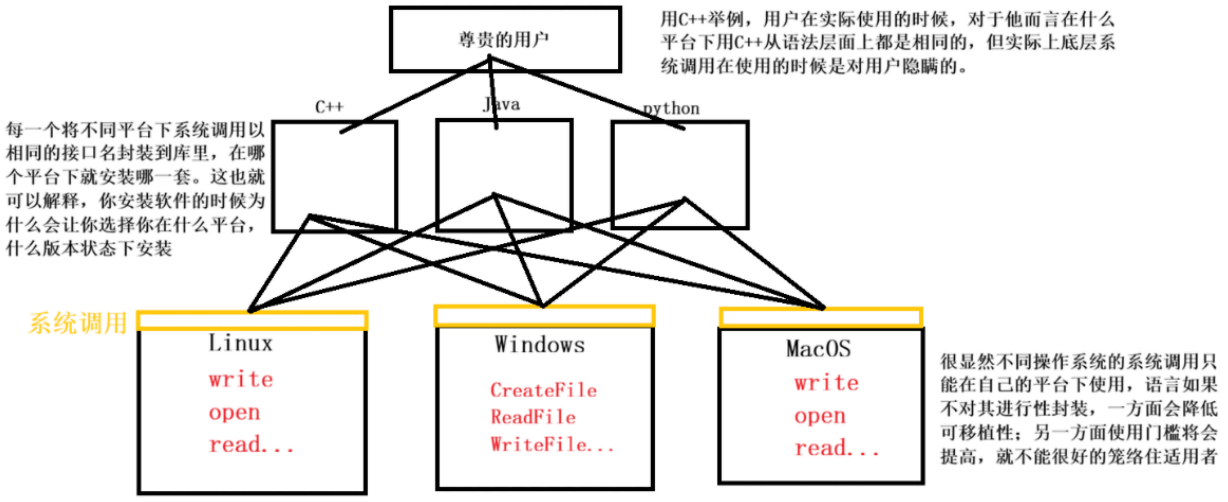


```
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ./openfile
fd: 3
fd1: 4
fd2: 5
```

1.4.4 语言库封装的意义：跨平台与效率

如果程序直接使用 Linux 的 `open`、`read`、`write`，这些接口在 Windows 上并不能原样使用。C 标准库为不同操作系统提供各自实现，但保留相对统一的函数接口，于是同一份 C 源码可以更方便地跨平台编译。

语言层封装还有效率意义：标准库可以在用户态缓存数据，合并多次小写入，减少进入内核的次数。



2. 文件描述符与进程打开文件表

2.1 文件描述符为什么是一个整数

2.1.1 `fd` 是进程文件描述符表中的下标

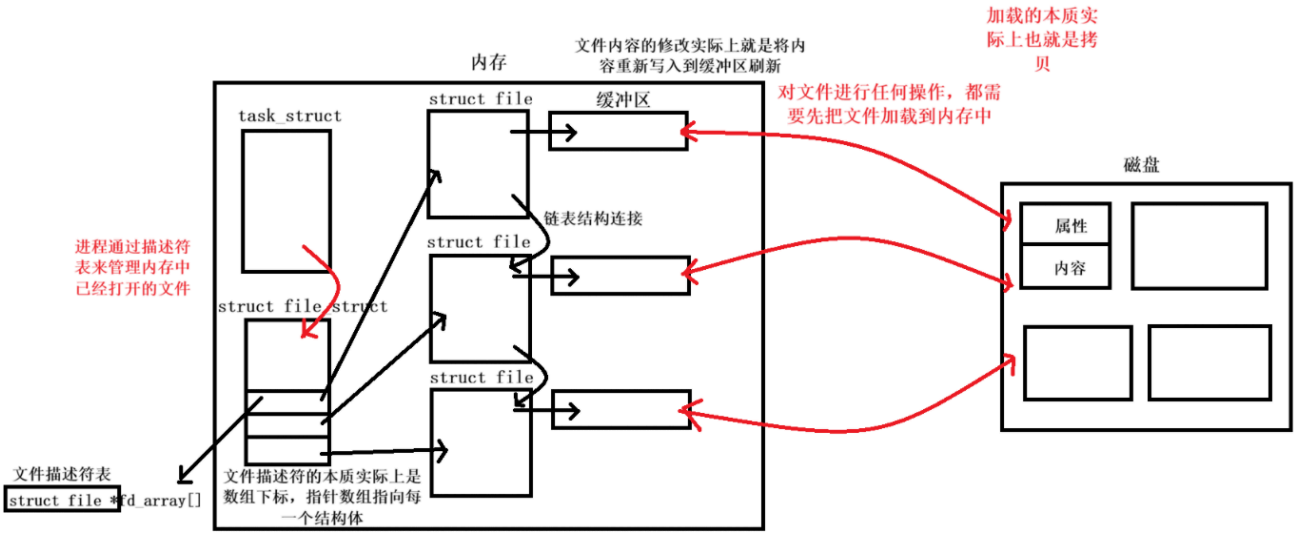
文件描述符不是磁盘地址，也不是文件内容的指针，而是一个小的非负整数。进程打开文件后，内核会在该进程的文件描述符表中分配一个表项，并把 `fd` 作为访问这个表项的编号。

整体关系可以简化为：

```
task_struct
    └── files_struct
            └── fd_array[]
```

```
                ├── [0] ──> struct file
                ├── [1] ──> struct file
                ├── [2] ──> struct file
                └── [3] ──> struct file
```

其中 `fd` 就是数组下标，数组表项指向内核中的打开文件描述对象。



2.1.2 进程与打开文件的关系

一个进程可以同时打开很多个文件，因此进程与打开文件之间是 `1:n` 的关系。为了管理这些打开文件，操作系统采用“先描述，再组织”的方法：

1. 用描述对象保存文件位置、打开模式、指向底层文件的引用等信息；
2. 用进程文件描述符表组织当前进程可访问的对象；
3. 用系统级结构记录多个进程可能共享的打开文件状态。

图示中的 `fd`、`FILE *` 和 `struct file` 处在不同层次：

- `fd` 是用户进程访问内核对象的整数编号；
- `FILE *` 是 C 库维护的用户态对象；
- `struct file` 是内核中的打开文件描述对象。

```
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ./openfile
fd: 3
```

2.2 文件描述符的分配规则

2.2.1 总是选择最小可用编号

如果 0、1、2 已经被占用，下一次 `open` 通常返回 3，之后是 4、5。

如果先关闭 0：

```
close(0);
int fd = open("log.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
printf("fd = %d\n", fd); // 通常为 0
```

系统会在文件描述符表中寻找最小且当前未使用的编号，因此新文件会填补刚空出的 0 号位置。

```
drwxrwxr-x  2 ZL ZL  4096 Jul 18 09:04 ./
drwxrwxr-x 18 ZL ZL  4096 Jul 18 08:55 ../
-rw-rw-r--  1 ZL ZL    70 Jul 18 08:55 Makefile
-rwxrwxr-x  1 ZL ZL 16048 Jul 18 09:04 openfile*
-rw-rw-r--  1 ZL ZL   409 Jul 18 09:04 openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ./openfile
fda: 0
fda: 3
fda: 4
```

2.2.2 三个默认描述符也遵循同一规则

0、1、2 之所以默认保留，是因为进程启动时标准输入、标准输出和标准错误已经打开。关闭其中任意一个，后续打开文件时就可能占用它的位置。

3. 重定向的本质

3.1 关闭后重新打开为什么能改变输出位置

3.1.1 `stdout` 只记住数字 1

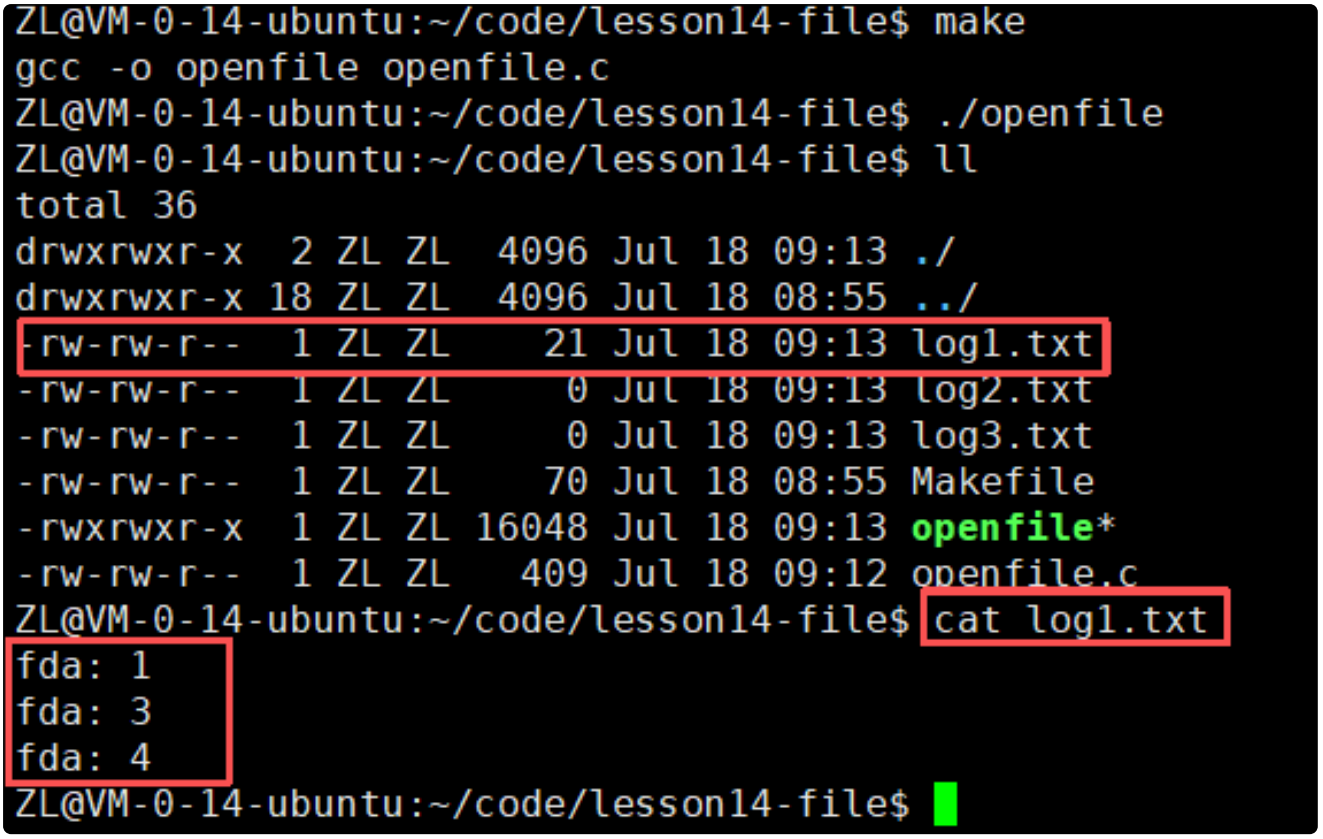
执行：

```
close(1);
int fd = open("log1.txt", O_WRONLY | O_CREAT | O_TRUNC, 0666);
printf("fd = %d\n", fd);
```

关闭 1 号描述符后，下一次 `open` 会优先得到 1。此后再调用 `printf`，`stdout` 仍然使用它内部记录的 `fd = 1`，但 1 号表项此刻已经指向 `log1.txt`。

所以，原本应该输出到显示器的数据被写进了文件，这就是**输出重定向**。

> 重定向的关键不是修改 `printf`，而是修改文件描述符编号所关联的目标。



3.1.2 覆盖与追加只相差打开标志

“覆盖重定向”使用 `>` 或 `O_TRUNC`；“追加重定向”使用 `>>` 或 `O_APPEND`。

```
close(1);
int fd = open("log1.txt",
              O_WRONLY | O_CREAT | O_APPEND,
              0666);
printf("append this line\n");
```

```
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ./openfile
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ cat log1.txt
fda: 1
fda: 1
fda: 1
fda: 1
fda: 1
fda: 1
```

3.2 用 `dup2` 复制文件描述符

3.2.1 `dup2` 建立两个编号到同一目标的关联

先关闭再重新打开虽然能演示重定向，但步骤较脆弱：如果 `open` 失败，标准输出就已经被破坏。

系统调用 `dup2` 可以把一个文件描述符关联到另一个编号：

```c
#include <unistd.h>

int dup2(int oldfd, int newfd);
```

执行 `dup2(oldfd, newfd)` 后：

1. 如果 `newfd` 已经打开，先关闭它；
2. 让 `newfd` 指向与 `oldfd` 相同的打开文件描述；
3. 成功时返回 `newfd`，失败时返回 `-1`；
4. 如果 `oldfd == newfd`，通常直接返回，不产生额外效果。

```
int dup2(int oldfd, int newfd);
```

3.2.2 输出重定向

```c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
```

```c
int main(void)
{
    int fd = open("log1.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0666);
    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    if (dup2(fd, 1) < 0)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    if (fd != 1)
        close(fd);

    printf("this line goes to log1.txt\n");
    return 0;
}
```

复制完成后，1 和 `fd` 都能访问同一个打开文件。若不再需要额外描述符，应关闭原来的 `fd`。

```
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ./openfile
fda: 0
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ll
total 36
drwxrwxr-x  2 ZL ZL  4096 Jul 18 10:50 ./
drwxrwxr-x 18 ZL ZL  4096 Jul 18 08:55 ../
-rw-rw-r--  1 ZL ZL    65 Jul 18 10:51 log1.txt
-rw-rw-r--  1 ZL ZL    70 Jul 18 08:55 Makefile
-rwxrwxr-x  1 ZL ZL 16216 Jul 18 10:50 openfile*
-rw-rw-r--  1 ZL ZL  1026 Jul 18 10:50 openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ cat log1.txt
hahahahahaha
hahahahahaha
hahahahahaha
hahahahahaha
hhhhhhhhhhh
ZL@VM-0-14-ubuntu:~/code/lesson14-file$
```

3.2.3 输入重定向

```
int fd = open("input.txt", O_RDONLY);
if (fd < 0)
{
    perror("open");
    return 1;
}

dup2(fd, 0);
char buffer[128];
fgets(buffer, sizeof(buffer), stdin);
```

此后 `stdin` 从文件读取，而不是从键盘读取。

```
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ll
total 36
drwxrwxr-x  2 ZL ZL  4096 Jul 18 10:08 ./
drwxrwxr-x 18 ZL ZL  4096 Jul 18 08:55 ../
-rw-rw-r--  1 ZL ZL    42 Jul 18 10:00 log1.txt
-rw-rw-r--  1 ZL ZL    70 Jul 18 08:55 Makefile
-rwxrwxr-x  1 ZL ZL 16096 Jul 18 10:08 openfile*
-rw-rw-r--  1 ZL ZL   714 Jul 18 10:08 openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ./openfile
asdfghj
asdfghj

ZL@VM-0-14-ubuntu:~/code/lesson14-file$
```

```
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ make
gcc -o openfile openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ./openfile
fda: 0
hhhhhhhhhhhhhhhhhhhhh redir stdin

ZL@VM-0-14-ubuntu:~/code/lesson14-file$
```

3.2.4 追加重定向

使用 `O_APPEND` 打开目标文件，再执行 `dup2(fd, 1)`，后续标准输出就会追加到文件末尾。

```
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ./openfile
fda: 0
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ll
total 36
drwxrwxr-x  2 ZL ZL  4096 Jul 18 10:58 ./
drwxrwxr-x 18 ZL ZL  4096 Jul 18 08:55 ../
-rw-rw-r--  1 ZL ZL    78 Jul 18 10:59 log1.txt
-rw-rw-r--  1 ZL ZL    70 Jul 18 08:55 Makefile
-rwxrwxr-x  1 ZL ZL 16128 Jul 18 10:58 openfile*
-rw-rw-r--  1 ZL ZL   939 Jul 18 10:58 openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ cat log1.txt
hahahahahaha
hahahahahaha
hahahahahaha
hahahahahaha
hhhhhhhhhhhh
lalalalalala
```

3.3 Shell 重定向与标准错误

3.3.1 Shell 先完成重定向，再执行目标程序

Shell 会先解析命令行中的重定向符号，打开相应文件并调整自己的子进程文件描述符，然后再执行目标程序。

常见写法：

```
./app >normal.txt        # 标准输出覆盖写入
./app >>normal.txt       # 标准输出追加
./app <input.txt         # 标准输入重定向
./app 2>err.txt          # 标准错误重定向
./app 1>normal.txt 2>err.txt
./app >all.txt 2>&1      # 标准输出和标准错误都写入 all.txt
```

`2>&1` 的顺序很重要。它表示让文件描述符 2 复制文件描述符 1 当前指向的目标。若写成 `2>&1 >all.txt`，2 会先复制旧的 1，随后 1 才被重定向，因此错误输出未必进入 `all.txt`。

3.3.2 标准错误存在的意义

标准输出适合正常结果，标准错误适合诊断信息。把二者分离后，可以只保存正常输出，或者把错误单独收集起来。

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void)
{
    const char *ok = "normal result\n";
    const char *err = "error message\n";

    write(1, ok, strlen(ok));
    write(2, err, strlen(err));
    perror("demo");
    return 0;
}
```

```
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$ vim text.c
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$ gcc text.c -o text
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$ ./text
这是一个正常消息
这也是一个正常的日志消息
这是一个正常消息，write
这是一个错误消息
这是一个错误消息，write
perror,hello: Success
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$ ./a.out > normal.txt
-bash: ./a.out: No such file or directory
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$ ./text > normal.txt
这是一个错误消息
这是一个错误消息，write
perror,hello: Success
```

```
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$ cat normal.txt
这是一个正常消息，write
这是一个正常消息
这也是一个正常的日志消息
```

```
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$ ./text 2>err.txt
这是一个正常消息
这也是一个正常的日志消息
这是一个正常消息，write
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$ cat err.txt
这是一个错误消息
这是一个错误消息，write
perror,hello: Success
ZL@VM-0-14-ubuntu:~/code/lesson16-mystdio/stderr$
```

4. 理解“一切皆文件”

4.1 统一抽象让不同设备拥有相似接口

4.1.1 普通文件与设备都可以被打开

在 Linux 中，普通文件、终端设备、管道、部分设备节点乃至套接字，都可以纳入文件描述符体系。用户进程仍然使用类似“打开、读取、写入、关闭”的方式访问它们。

“一切皆文件”描述的是接口和对象模型上的统一，不代表磁盘、网卡和管道在物理上真的都是普通磁盘文件。

4.1.2 内核用 `struct file` 描述一次打开关系

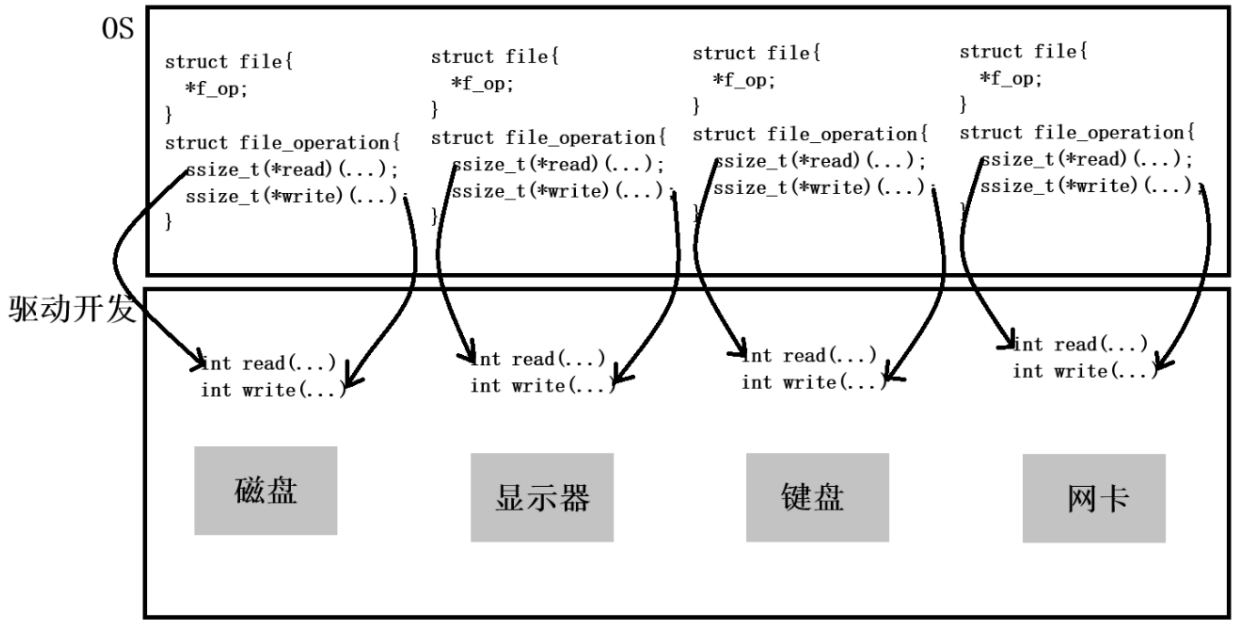
内核中的打开文件描述对象会记录访问状态，并关联底层文件或设备的信息。不同设备的实际读写方法可以挂在操作函数表中。

概念上可以写成：

```
struct file
    ├── 打开标志、当前位置、访问模式等属性
    └── 指向文件操作函数的 f_op
```

当进程对 `fd` 发起 `read` 或 `write` 时，内核会找到对应的打开文件描述，再调用该对象关联的具体读写方法。

从 C 语言角度看，这类似于用结构体保存函数指针，再在运行时按对象类型调用不同函数，因此常把它理解为“C 语言模拟多态”。



5. 缓冲区的完整数据路径

5.1 一次 I/O 为什么会有多次数据拷贝

5.1.1 读取路径

读取文件时，数据通常先由存储设备进入内核页缓存，再通过 `read` 从内核空间拷贝到用户提供的缓冲区：

```
磁盘/存储设备
    -> 内核页缓存
    -> read(fd, user_buffer, n)
    -> 进程用户缓冲区
```

如果所需数据尚未在内核缓存中，进程可能需要等待内核从外部设备取回数据。

5.1.2 写入路径

写入文件时，数据首先从用户空间交给内核，进入对应的内核页缓存，随后由操作系统在合适时机写回外部存储：

```
进程用户缓冲区
    -> write(fd, user_buffer, n)
    -> 内核页缓存
    -> 操作系统刷新
    -> 存储设备
```

所以，`write` 成功通常表示数据已经交给内核，但不一定表示数据已经永久写到了磁盘。真正要求持久化时，还需要使用 `fsync` 等同步接口。

5.1.3 修改文件也要遵循“加载、修改、刷新”

文件 I/O 的常见过程是：

1. 先把所需内容加载到内存缓冲区；
2. 在内存中修改；
3. 再把修改后的内容刷新回内核或存储设备。

系统调用 `lseek` 可以修改打开文件的当前位置。它只改变后续读写从哪里开始，并不直接读写文件内容。

```
off_t old = lseek(fd, 0, SEEK_CUR);
lseek(fd, 0, SEEK_SET);
lseek(fd, 0, SEEK_END);
```

```
lseek(2)

NAME
       lseek - reposition read/write file offset

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       off_t lseek(int fd, off_t offset, int whence);
```

5.1.4 用户缓冲区由程序自己提供

```
char buffer[1024];
ssize_t n = read(fd, buffer, sizeof(buffer) - 1);
if (n > 0)
{
    buffer[n] = '\0';
    printf("%s\n", buffer);
}
```

这里的 `buffer` 位于进程的用户空间，程序可以直接通过指针访问它。`read` 的一次调用就涉及一次用户态与内核态之间的边界，因此系统调用本身有成本。

```
ZL@VM-0-14-ubuntu:~/code/lesson15-file$ ./readfile
buffer: dfghjklll
sdfghjkl
```

5.2 用户缓冲区、语言级缓冲区与内核缓冲区

5.2.1 三种缓冲区的区别

层次	所在位置	由谁管理	作用
用户缓冲区	进程用户空间	应用程序	暂存本次读写的数据
语言级缓冲区	`FILE` 对象中	C 标准库等运行时	合并多次小规模读写，减少系统调用
内核页缓存	内核空间	操作系统	缓存文件数据并组织磁盘 I/O

可以用一条路径表示：

```
程序数据
    -> FILE 语言级缓冲区
    -> write(fd)
    -> 内核页缓存
    -> 存储设备
```

5.2.2 为什么要减少系统调用

系统调用需要切换到内核态，完成参数检查和上下文切换，再返回用户态。一次只写几个字节就调用一次 `write`，大量小写入会把成本放大。

C 标准库因此为 `FILE` 维护输出缓冲区，让多个小写入先在用户态积累，等满足条件后再统一调用 `write`。

这也是 `fputs` 在连续小写入场景下通常比直接循环调用 `write` 更高效的原因。它不是绝对“永远更快”，而是通过批处理减少系统调用次数。

5.2.3 `fopen` 与 `fclose` 在缓冲链路中的位置

可以粗略理解为：

1. `fopen` 调用底层 `open` 获得文件描述符，并创建 `FILE` 对象；
2. 后续语言层读写先操作 `FILE` 缓冲区；
3. `fclose` 刷新尚未写出的数据，关闭文件描述符，并释放 `FILE` 对象。

C++ 的 `ifstream`、`ofstream` 等流对象也承担类似职责：底层依赖操作系统文件描述符，外层维护缓冲和状态。

5.3 标准 I/O 的三种缓冲模式

5.3.1 全缓冲

缓冲区满了以后才批量调用系统调用。普通磁盘文件通常采用全缓冲。

5.3.2 行缓冲

遇到换行符或缓冲区满时刷新。终端上的标准输出通常采用行缓冲，因此带换行的日志能及时显示。

5.3.3 无缓冲

库不缓存数据，直接触发系统调用。标准错误 `stderr` 通常采用无缓冲模式，使错误信息尽快显示。

具体缓冲区大小和默认模式与 C 库实现、终端类型和运行环境有关，不应把某个固定的 1024 字节当成所有系统的保证。

5.3.4 常见刷新时机

语言级缓冲区可能在以下情况下刷新：

1. 缓冲区已经写满；
2. 行缓冲遇到换行符；
3. 显式调用 `fflush`；
4. 正常调用 `fclose`；
5. 程序通过正常出口结束，运行时清理标准流。

`fflush(stdout)` 会尝试把语言级缓冲区中的数据通过底层写入接口交给内核，但它通常不等同于 `fsync`，不保证数据已经写进持久化存储。

5.4 三个典型实验

5.4.1 只关闭 fd，没有刷新语言缓冲区

```c
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    close(1);
    int fd = open("log.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0666);
    if (fd < 0)
        return 1;

    printf("fd = %d\n", fd);
    printf("hello printf\n");

    close(fd);
    return 0;
}
```

这段代码中，`printf` 的数据可能仍停留在 `stdout` 的语言级缓冲区里。`close(fd)` 直接关闭的是文件描述符，没有先刷新 `stdout`。进程正常退出时，运行时再尝试刷新，但 1 号描述符已经不可用，数据就可能丢失。

正确做法之一是在关闭底层描述符前显式刷新：

```c
fflush(stdout);
close(fd);
```

实际工程中更推荐直接使用 `fclose(stdout)`，或者避免手工关闭标准描述符来模拟重定向。

```
drwxrwxr-x  2 ZL ZL 4096 Jul 18 09:39 ./
drwxrwxr-x 18 ZL ZL 4096 Jul 18 08:55 ../
-rw-rw-r--  1 ZL ZL   70 Jul 18 08:55 Makefile
-rw-rw-r--  1 ZL ZL  508 Jul 18 09:37 openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ make
gcc -o openfile openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ./openfile
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ ll
total 32
drwxrwxr-x  2 ZL ZL  4096 Jul 18 09:39 ./
drwxrwxr-x 18 ZL ZL  4096 Jul 18 08:55 ../
-rw-rw-r--  1 ZL ZL     0 Jul 18 09:39 log1.txt
-rw-rw-r--  1 ZL ZL    70 Jul 18 08:55 Makefile
-rwxrwxr-x  1 ZL ZL 16048 Jul 18 09:39 openfile*
-rw-rw-r--  1 ZL ZL   508 Jul 18 09:37 openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson14-file$ cat log1.txt
ZL@VM-0-14-ubuntu:~/code/lesson14-file$
```

5.4.2 `_exit` 会跳过标准 I/O 清理

```c
#include <unistd.h>

int main(void)
{
    printf("hello world");
    _exit(0);
}
```

`_exit` 直接终止进程，不执行 C 运行时正常的 `exit` 清理流程，因此尚未刷新的语言级缓冲区数据可能消失。

与它相比，从 `main` 正常 `return 0` 或调用 `exit(0)`，通常会执行标准 I/O 清理并刷新相关流。

5.4.3 `fork` 后为什么会出现重复输出

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
```

```c
int main(void)
{
    printf("hello printf\n");
    fprintf(stdout, "hello fprintf\n");
    fputs("hello fputs\n", stdout);
    write(1, "hello write\n", strlen("hello write\n"));

    fork();
    return 0;
}
```

需要区分两种运行环境：

1. 直接输出到终端时，`stdout` 通常是行缓冲。三条带换行的 C 库输出在 `fork` 前已经刷新，因此父进程和子进程不会再重复持有这些数据。
2. 重定向到普通文件时，`stdout` 通常变成全缓冲。在 `fork` 前，三条 C 库输出可能仍留在 `FILE` 缓冲区中。`fork` 后父子进程各自拥有一份地址空间副本，退出时都会刷新这份缓冲区，于是三条 C 库输出各出现两次。
3. `write` 是在 `fork` 前直接完成的系统调用。数据早已交给内核，因此只出现一次。

所以重定向到文件后，可能看到 3 条 C 库输出各重复一次，再加上 1 条 `write` 输出，共 7 行。

> 这个实验说明：缓冲区属于进程用户态运行时的一部分，不属于内核文件数据本身。

```
ZL@VM-0-14-ubuntu:~/code/lesson15-file$ ./readfile
hello printf
hello fprintf
hello fputs
hello write
ZL@VM-0-14-ubuntu:~/code/lesson15-file$ ./readfile > log.txt
ZL@VM-0-14-ubuntu:~/code/lesson15-file$ cat log.txt
hello write
hello printf
hello fprintf
hello fputs
hello printf
hello fprintf
hello fputs
ZL@VM-0-14-ubuntu:~/code/lesson15-file$
```

5.5 从内核缓冲区到持久化存储

5.5.1 `fflush` 与 `fsync` 不是同一层操作

`fflush` 刷新的是 C 标准库的用户态流缓冲；`fsync` 面向文件描述符，要求内核把文件数据和必要元数据同步到存储设备。

```c
#include <fcntl.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

int main(void)
{
    int fd = open("log.txt", O_WRONLY | O_CREAT | O_APPEND, 0666);
    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    const char *msg = "durable\n";
    ssize_t n = write(fd, msg, strlen(msg));
    if (n == (ssize_t)strlen(msg))
        fsync(fd); // 请求内核把该文件状态真正同步到存储设备

    close(fd);
    return 0;
}
```

这两个调用解决的是不同问题：

- `fflush` 解决“C 库缓冲区还没有交给内核”；
- `fsync` 解决“内核缓存还没有稳定写回存储设备”。

`fdatasync` 与 `fsync` 类似，但更关注文件数据以及完成这些数据访问所必需的元数据。文件系统、存储设备缓存和硬件行为仍会影响最终持久化的实现细节。

```
NAME
       fsync, fdatasync - synchronize a file's in-core state with storage device

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int fsync(int fd);

       int fdatasync(int fd);
```

6. 手写一个精简版 `stdio`

6.1 设计目标

6.1.1 用 `myFILE` 复现“描述符 + 缓冲区”

为了让缓冲区理论落到代码，可以封装一个极简的 `myFILE`。它只保留实现演示所需的核心成员：

```c
typedef struct myFILE
{
    int fd;
    int flags;
    int flush_mode;
    char outbuffer[1024];
    size_t pos;
    size_t cap;
} myFILE;
```

其中：

- `fd` 指向内核中的打开文件；
- `outbuffer` 是语言级缓冲区；
- `pos` 表示当前已经缓存多少字节；
- `flush_mode` 用位标志表示刷新策略。

6.1.2 头文件 `mystdio.h`

```h
#pragma once

#include <stddef.h>

#define NON_BUFFER  1
#define LINE_BUFFER 2
#define FULL_BUFFER 4
#define BUFFER_SIZE 1024
#define MODE 0666

typedef struct myFILE
{
    int fd;
    int flags;
    int flush_mode;
    char outbuffer[BUFFER_SIZE];
    size_t pos;
    size_t cap;
} myFILE;

myFILE *myfopen(const char *pathname, const char *mode);
int myfputs(const char *str, myFILE *stream);
int myfflush(myFILE *stream);
int myfclose(myFILE *stream);
```

6.2 实现核心接口

6.2.1 打开文件

```c
#include "mystdio.h"

#include <fcntl.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

myFILE *myfopen(const char *pathname, const char *mode)
{
    int flags;

    if (strcmp(mode, "r") == 0)
        flags = O_RDONLY;
    else if (strcmp(mode, "w") == 0)
```

```
        flags = O_WRONLY | O_CREAT | O_TRUNC;
    else if (strcmp(mode, "a") == 0)
        flags = O_WRONLY | O_CREAT | O_APPEND;
    else
        return NULL;

    int fd = open(pathname, flags, MODE);
    if (fd < 0)
        return NULL;

    myFILE *stream = malloc(sizeof(*stream));
    if (stream == NULL)
    {
        close(fd);
        return NULL;
    }

    stream->fd = fd;
    stream->flags = flags;
    stream->flush_mode = LINE_BUFFER;
    stream->pos = 0;
    stream->cap = BUFFER_SIZE;
    return stream;
}
```

6.2.2 刷新缓冲区

这里忽略“`write` 可能部分写入”等生产级细节，只展示核心流程：把用户态缓冲区中的数据交给内核。

```
static int flush_all(myFILE *stream)
{
    if (stream->pos == 0)
        return 0;

    ssize_t n = write(stream->fd, stream->outbuffer, stream->pos);
    if (n < 0 || (size_t)n != stream->pos)
        return -1;

    stream->pos = 0;
    return 0;
}

static int try_flush(myFILE *stream)
```

```
{
    if (stream->flush_mode & NON_BUFFER)
        return flush_all(stream);

    if (stream->flush_mode & LINE_BUFFER)
    {
        if (stream->pos > 0 &&
            stream->outbuffer[stream->pos - 1] == '\n')
            return flush_all(stream);
    }

    if (stream->flush_mode & FULL_BUFFER)
    {
        if (stream->pos >= stream->cap)
            return flush_all(stream);
    }

    return 0;
}

int myfflush(myFILE *stream)
{
    return flush_all(stream);
}
```

6.2.3 写入数据

```
int myfputs(const char *str, myFILE *stream)
{
    size_t len = strlen(str);
    if (len == 0)
        return 0;

    if (len > stream->cap - stream->pos)
    {
        if (flush_all(stream) < 0)
            return -1;
    }

    if (len > stream->cap)
    {
        ssize_t n = write(stream->fd, str, len);
        return n == (ssize_t)len ? (int)len : -1;
    }
```

```
    memcpy(stream->outbuffer + stream->pos, str, len);
    stream->pos += len;

    if (try_flush(stream) < 0)
        return -1;

    return (int)len;
}
```

6.2.4 关闭文件

```
int myfclose(myFILE *stream)
{
    int result = 0;

    if (flush_all(stream) < 0)
        result = -1;

    if (close(stream->fd) < 0)
        result = -1;

    free(stream);
    return result;
}
```

这里没有强制调用 `fsync`。普通 `fclose` 的主要语义是完成标准库清理并关闭描述符；“立即同步到存储设备”是额外的持久化要求，不是每次关闭文件都必须承担的操作。

6.3 验证缓冲区行为

```
#include "mystdio.h"

#include <stdio.h>
#include <unistd.h>

int main(void)
{
    myFILE *fp = myfopen("log.txt", "a");
    if (fp == NULL)
    {
        perror("myfopen");
```

```
        return 1;
    }

    for (int i = 0; i < 10; ++i)
    {
        myfputs("hello", fp);
        printf("cached bytes: %zu\n", fp->pos);
        sleep(1);
    }

    myfclose(fp);
    return 0;
}
```

程序运行期间可以观察 `pos` 的变化。在当前行缓冲策略下，不带换行的 `"hello"` 会先停留在 `outbuffer` 中，直到缓冲区需要刷新或 `myfclose` 被调用。

这个精简实现只用于理解原理。生产级标准库还要处理：

1. 读写混合、文件定位与同步；
2. 部分写入和可中断系统调用；
3. 输入缓冲与输出缓冲；
4. 线程安全、错误状态和格式化输出；
5. 更多打开模式、编码和平台差异。

7. 本章总结

1. C 标准库文件接口最终依赖 Linux 系统调用，库函数在系统调用外增加了缓冲、状态与跨平台封装。
2. 文件描述符是进程文件描述符表中的下标；打开文件的内核状态由描述对象保存，不能把 `fd` 理解为磁盘地址。
3. 新文件描述符优先使用最小可用编号，0、1、2 默认对应标准输入、标准输出和标准错误。
4. 输出重定向的本质是让文件描述符 1 关联到新的打开文件；`dup2` 可以安全地把两个描述符关联到同一打开文件描述。
5. “一切皆文件”是统一的访问抽象，底层通过文件对象及其操作函数适配不同设备。
6. 数据可能依次经过用户缓冲区、`FILE` 语言级缓冲区和内核页缓存，多层缓冲共同减少系统调用和慢速设备访问。
7. `fflush` 把语言级缓冲交给内核，`fsync` 才进一步要求内核把文件状态同步到存储设备。